

# Manejo e Implementación de Archivos



Semana #8

Guatemala, 26 de agosto de 2026

Ing. Luis Aguilar



# Objetivos

---

Al finalizar la clase el estudiante será capaz de:

Comprender el concepto de integridad de datos y su importancia en los archivos.

Diferenciar corrupción accidental de modificación intencional de información.

Identificar cambios en archivos mediante huellas digitales (hash).

Aplicar conceptos de integridad de datos en casos prácticos de manejo de archivos.

# 1 INTEGRIDAD DE DATOS

## ¿CÓMO DEMOSTRAR QUE UN ARCHIVO NO CAMBIÓ?

**Objetivo:** Comprender qué es la integridad de datos y su importancia en el manejo de archivos.

### ¿QUÉ ES INTEGRIDAD?



Garantiza que la información se mantenga exacta, completa y sin cambios no autorizados durante su ciclo de vida:



CREACIÓN



ALMACENAMIENTO



TRANSMISIÓN



USO

### PREGUNTA CLAVE



¿Cómo puedo demostrar que un archivo no cambió desde que fue creado o enviado?

### EJEMPLOS COTIDIANOS



#### XML MODIFICADO

Se cambia un dato en el XML y el archivo sigue pareciendo válido, pero ya no es el original.



#### DESCARGA INCOMPLETA

La descarga se interrumpe y el archivo queda incompleto o corrupto.



#### DOCUMENTO ALTERADO

Se modifica un documento de texto cambiando una palabra o cifra importante.



#### CONFIGURACIÓN CAMBIADA

Se modifica un archivo de configuración y el sistema puede comportarse de forma incorrecta.

### CORRUPCIÓN VS. MODIFICACIÓN INTENCIONAL



#### CORRUPCIÓN ACCIDENTAL

Cambios no deseados causados por errores del sistema, fallos de hardware, interrupciones o transferencias incompletas.



#### MODIFICACIÓN INTENCIONAL

Cambios realizados a propósito por un usuario o atacante para alterar la información.



### ¿POR QUÉ ES IMPORTANTE?



Asegura que la información sea confiable.



Permite detectar cambios no autorizados.



Protege contra fraudes y manipulaciones.



Garantiza la calidad de los datos.



Evita errores en procesos por datos incorrectos.

### EN RESUMEN



ARCHIVO ORIGINAL



HUELLA DIGITAL (HASH)



SE GUARDA LA HUELLA



SE USA O SE TRANSMITE



SE CALCULA NUEVAMENTE



COINCIDEN  
Archivo íntegro



NO COINCIDEN  
Archivo alterado



LA INTEGRIDAD DE DATOS RESPONDE A LA PREGUNTA: "¿PUEDO CONFIAR EN QUE ESTE ARCHIVO ES EXACTAMENTE EL MISMO QUE EL ORIGINAL?"



# 2 HASHING BÁSICO

## ¿CÓMO FUNCIONA?

Las funciones hash convierten cualquier archivo en una **huella digital única**.



### IDEA CLAVE

Si el archivo cambia aunque sea un solo carácter, la huella cambia **por completo**.

#### 1. ARCHIVO ORIGINAL



VS

#### 2. ARCHIVO MODIFICADO (1 CARÁCTER)



### PREGUNTA PARA REFLEXIONAR

¿Por qué cambió completamente la huella si solamente cambiamos un carácter?



Coméntalo  
con tus compañeros

### ¿QUÉ ES UNA FUNCIÓN HASH?

Es un algoritmo que convierte datos de cualquier tamaño en una cadena de longitud fija (la huella).



### PROPIEDADES PRINCIPALES



#### Determinista

Mismos datos siempre generan la misma huella.



#### Única

Pequeños cambios producen huellas completamente diferentes.



#### Unidireccional

No es posible obtener los datos originales a partir de la huella.

### EFECTO AVALANCHA

Un cambio mínimo en la entrada produce un cambio drástico e impredecible en la salida.



### ALGORITMOS COMUNES

MD5

SHA-1

SHA-256

SHA-512

### ¿PARA QUÉ SE USA?



Verificar integridad de archivos



Verificar descargas



Firmas digitales



Almacenamiento seguro de contraseñas

### EN RESUMEN



Hashing nos permite verificar que un archivo no ha sido alterado, de manera rápida y confiable.

## 3

# HUELLA DIGITAL DE ARCHIVOS

## ¿CÓMO VERIFICAR LA INTEGRIDAD?

La huella digital (hash) permite comprobar si un archivo se mantiene íntegro o ha sido alterado.



### ¿QUÉ ES LA HUELLA DIGITAL?

Es una cadena única de caracteres generada a partir de un archivo. Si el archivo cambia, la huella cambia.



### RESULTADO DE LA COMPARACIÓN



#### ARCHIVO ÍNTEGRO

Hash original = Hash actual  
El archivo **NO** ha sido modificado.



#### ARCHIVO MODIFICADO

Hash original  $\neq$  Hash actual  
El archivo **SÍ** ha sido modificado.



### ¿PARA QUÉ SIRVE?



Detectar cambios accidentales o maliciosos.



Verificar integridad en descargas o transferencias.



Asegurar que el archivo es exactamente el mismo que el original.



Garantizar la confiabilidad de la información.



### RECUERDA

No importa el tamaño del archivo: cualquier cambio, por mínimo que sea, generará una huella digital completamente diferente.



## 4

# DEMOSTRACIÓN EN VIVO

## VERIFICACIÓN DE INTEGRIDAD DE UN ARCHIVO CON POWERSHELL

**Objetivo:** Entender cómo una huella digital (SHA-256) permite detectar cambios en un archivo.



### IDEA CLAVE

Si el archivo cambia, aunque sea un solo carácter, la huella digital cambia por completo.

#### 1 CREAMOS EL ARCHIVO

Creamos un archivo XML llamado datos.xml.



datos.xml

```
<alumnos>
<alumno id="1">
  Juan Pérez
</alumno>
<alumno id="2">
  María López
</alumno>
</alumnos>
```

#### 2 CALCULAMOS LA HUELLA (SHA-256)

Abrimos PowerShell en la carpeta y calculamos la huella del archivo.



Comando:

```
Get-FileHash .\datos.xml
-Algorithm SHA256
```

Resultado en pantalla:

| Algorithm | Hash                    |
|-----------|-------------------------|
| SHA256    | A3F5C8D9E7B1F4A2...91D2 |

#### 3 GUARDAMOS LA HUELLA

Guardamos el hash como referencia para futuras verificaciones.



hash.txt

Comando:

```
Get-FileHash .\datos.xml
-Algorithm SHA256 |
Out-File .\hash.txt
```

Contenido de hash.txt:

```
A3F5C8D9E7B1F4A2...91D2
```

#### 4 MODIFICAMOS EL ARCHIVO

Cambiamos manualmente un solo carácter en datos.xml.



Ejemplo:

Cambiamos:

Juan Pérez

por:

Juan Perez

(se quitó la "é")

#### 5 CALCULAMOS NUEVAMENTE LA HUELLA

Volvemos a ejecutar el comando en PowerShell.



Comando:

```
Get-FileHash .\datos.xml
-Algorithm SHA256
```

Resultado en pantalla:

| Algorithm | Hash                    |
|-----------|-------------------------|
| SHA256    | B7210AC6F9D3C0E8...C821 |

#### 6 COMPARAMOS LAS HUELLAS

Comparamos el hash actual con el registrado.



Huella registrada:

```
A3F5C8D9E7B1F4A2...91D2
```

≠

Huella actual:

```
B7210AC6F9D3C0E8...C821
```

### RESULTADO EN CONSOLA (EJEMPLO)

```
=====
VERIFICACIÓN DE INTEGRIDAD
=====
Archivo: datos.xml
Hash registrado: A3F5C8D9E7B1F4A2...91D2
Hash actual: B7210AC6F9D3C0E8...C821
RESULTADO: ! ARCHIVO ALTERADO
```

### ¿QUÉ OBSERVAMOS?



Solo modificamos un carácter.



La huella digital cambió completamente.



El hash permite detectar cambios intencionales o accidentales.



Esto garantiza la integridad de la información.

### PARA ENTENDERLO MEJOR

Archivo original  
(como una receta)



Huella digital  
(como la firma de la receta)



Si alguien cambia un ingrediente, la firma ya no coincide.



### ACTIVIDAD PARA EL ESTUDIANTE

- 1 Crea tu propio archivo XML.
- 2 Calcula su huella con PowerShell.
- 3 Guarda la huella en hash.txt.
- 4 Modifica un dato del XML.
- 5 Calcula nuevamente la huella.
- 6 Compara ambas huellas.
- 7 Explica por qué el resultado cambió.

### COMANDO POWERSHELL CLAVE

Calcula la huella SHA-256 de un archivo:

```
Get-FileHash .\nombrearchivo -Algorithm SHA256
```

Guarda la huella en un archivo de texto:

```
Get-FileHash .\nombrearchivo -Algorithm SHA256 |
Out-File .\hash.txt
```

# 5 HASH vs. CIFRADO

¿En qué se parecen y en qué son diferentes?

Ambos transforman información, pero tienen **objetivos distintos** y se usan para **cosas diferentes**.

|                                                                                                               | HASH                                                                                                                                                                                                                                         | CIFRADO                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>OBJETIVO PRINCIPAL</b>   |  <b>Verifica integridad</b><br>Permite comprobar si los datos han cambiado.                                                                                 |  <b>Protege confidencialidad</b><br>Impide que otras personas puedan leer la información.                                |
|  <b>¿SE PUEDE RECUPERAR?</b> |  <b>No está diseñado para recuperarse</b><br>A partir del hash, no se puede obtener el archivo original.                                                    |  <b>Se puede descifrar con una clave</b><br>Con la clave correcta, se puede recuperar la información original.           |
|  <b>ALGORITMO EJEMPLO</b>    |  <b>SHA-256</b><br>Función hash criptográfica utilizada ampliamente.                                                                                        |  <b>AES</b><br>Algoritmo de cifrado simétrico muy utilizado.                                                             |
|  <b>¿QUÉ HACE?</b>           |                                                                                                                                                            |                                                                                                                          |
|  <b>¿PARA QUÉ SE USA?</b>  | <ul style="list-style-type: none"><li>• Detectar cambios en archivos.</li><li>• Verificar descargas (integridad).</li><li>• Firmas digitales (como parte del proceso).</li><li>• Almacenar contraseñas (con técnicas adicionales).</li></ul> | <ul style="list-style-type: none"><li>• Proteger información confidencial.</li><li>• Cifrar archivos y comunicaciones.</li><li>• Proteger contraseñas, documentos, bases de datos, mensajes, etc.</li></ul> |



## PREGUNTA PARA LOS ALUMNOS

Si hago SHA-256 de un archivo, ¿puedo recuperar el archivo original a partir del hash?



## RESPUESTA: No.

El hash es una huella digital unidireccional: está diseñado para verificar, no para reconstruir el original.



## RECUERDA

Hash = verificar integridad  
Cifrado = proteger información  
Son herramientas diferentes para necesidades diferentes.



# ¿POR QUÉ SHA-256 Y NO MD5?

Comparación de algoritmos hash criptográficos

| COMPARACIÓN RÁPIDA |                   |                  |                                    |                                             |
|--------------------|-------------------|------------------|------------------------------------|---------------------------------------------|
| ALGORITMO          | LONGITUD DEL HASH | SEGURIDAD ACTUAL | COLISIONES CONOCIDAS               | USO RECOMENDADO                             |
| MD5                | 128 bits          | Débil            | Sí, muchas colisiones prácticas    | No recomendado para seguridad               |
| SHA-1              | 160 bits          | Débil            | Sí, colisiones prácticas           | No recomendado para seguridad               |
| SHA-256            | 256 bits          | Fuerte           | No se conocen colisiones prácticas | Muy recomendado para integridad             |
| SHA-512            | 512 bits          | Muy fuerte       | No se conocen colisiones prácticas | Recomendado para altos niveles de seguridad |

**DEMOSTRACIÓN EN POWERSHELL**

El mismo archivo produce huellas diferentes según el algoritmo usado.

Archivo de ejemplo: `datos.xml`

```
# Ejecuta cada comando en PowerShell
Get-FileHash .\datos.xml -Algorithm MD5
Get-FileHash .\datos.xml -Algorithm SHA1
Get-FileHash .\datos.xml -Algorithm SHA256
Get-FileHash .\datos.xml -Algorithm SHA512
```

Ejemplo de salida en PowerShell

| Algorithm | Hash                                                                                                              |
|-----------|-------------------------------------------------------------------------------------------------------------------|
| MD5       | 7B2C1A9F3E4D5C6B8A7D9E0F12345678                                                                                  |
| SHA1      | 4A7D1ED414474E4033AC29CC3D9F5C6A                                                                                  |
| SHA256    | A3F5C8D9E7B1F4A2C6D3E8F9A1B2C3D45E6F7A8B9C0D1E2F3A4B5C6D7E8F9A0B                                                  |
| SHA512    | 8C3F2D1A7B9E4C6D5F0A1B2C3D4E5F6A7B8C9D0E1F2A3B4C5D6E7F8091A2B3C4D5E6F708192A3B4C5D6E7F8091A2B3C4D5E6F708192A3B4C5 |

Observa cómo cada algoritmo genera una huella (hash) de longitud diferente para el mismo archivo.

**¿QUÉ SIGNIFICA ESTO?**

- MD5** y **SHA-1** ya no son seguros: se han encontrado colisiones (dos entradas diferentes que producen el mismo hash).
- SHA-256** es una opción habitual y confiable para verificar integridad de archivos y datos.
- Una mayor longitud (más bits) ofrece más espacio para valores únicos, pero no significa automáticamente que un algoritmo sea “mejor” para todos los usos.
- Para integridad de archivos, **SHA-256** es una elección práctica, segura y ampliamente utilizada.

**LONGITUD DEL HASH (EN BITS)**

|         |  |          |
|---------|--|----------|
| MD5     |  | 128 bits |
| SHA-1   |  | 160 bits |
| SHA-256 |  | 256 bits |
| SHA-512 |  | 512 bits |

Más bits = más combinaciones posibles = mayor resistencia a ataques de fuerza bruta.

**EN RESUMEN**

**MD5 y SHA-1**  
Ya no deben usarse para seguridad. Existen ataques reales que permiten encontrar colisiones.

**SHA-256**  
Excelente equilibrio entre seguridad, rendimiento y compatibilidad. Estándar ampliamente adoptado.

**SHA-512**  
Ofrece mayor longitud y seguridad, útil en contextos que requieren máxima protección.

**No siempre “más bits” es “mejor”**  
Depende del uso, del contexto y de los requisitos de seguridad y rendimiento.





# ¿DÓNDE SE UTILIZA REALMENTE?

La huella digital (hash) con SHA-256 se usa en muchas situaciones reales para garantizar que la información no ha sido alterada.

## 1. ARCHIVOS DESCARGADOS

Verifica que el archivo descargado es idéntico al original.

EN EL SERVIDOR (ORIGINAL)



Archivo original

SHA-256

A8F92...

EN TU EQUIPO (DESCARGADO)



Archivo descargado

SHA-256

A8F92...



Coinciden → probablemente recibiste exactamente el mismo contenido.

## 2. INSTALADORES Y PROGRAMAS

Los desarrolladores publican el hash para que verifiques el instalador.



Descargas el instalador del sitio oficial

SHA-256

ABC123...



El desarrollador publica el hash oficial

ABC123...



Si coinciden, el instalador es auténtico y no fue modificado.

## 3. RESPALDOS (BACKUPS)

Verificas que tus respaldos no hayan sido corrompidos o modificados.



Respaldo guardado en el pasado

SHA-256

7C59B...



Respaldo verificado actualmente



Si coinciden, el respaldo está íntegro.

## 4. ARCHIVOS DE CONFIGURACIÓN

Detecta cambios no autorizados en archivos críticos del sistema o aplicaciones.



config.json (original)

SHA-256

9B12A...



config.json (modificado)



Si no coinciden, el archivo fue alterado.

## 5. INTEGRIDAD EN COMUNICACIONES

Asegura que los datos enviados por la red no fueron alterados.



Datos enviados

SHA-256

D4E55...



Datos recibidos



Si coinciden, los datos llegaron intactos.

## ¿POR QUÉ ES IMPORTANTE?



- Garantiza integridad de la información.
- Detecta cambios accidentales o maliciosos.
- Genera confianza en archivos, sistemas y procesos.
- Es rápido, fácil de usar y ampliamente soportado.

## ¿CÓMO FUNCIONA?



Archivo

Algoritmo SHA-256



Huella digital (hash)

Se compara con la huella esperada



## RECUERDA

El hash no cifra ni oculta la información, solo crea una "firma digital" única del contenido. Si algo cambia, la huella cambia por completo.



# SSH Y CLONADO DE REPOSITORIOS DE GITHUB

SSH te permite conectarte de forma segura a servidores remotos y clonar repositorios sin usar contraseña cada vez.



## ¿QUÉ ES SSH?

SSH (Secure Shell) es un protocolo que permite conectarse de forma segura a servidores remotos.

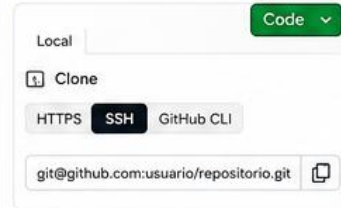
- ✓ Cifra la comunicación.
- ✓ Usa un par de claves: pública y privada.
- ✓ Autentica tu identidad sin enviar contraseñas.
- ✓ Ideal para trabajar con Git y GitHub.

## ¿PARA QUÉ SIRVE?

- 📁 Clonar repositorios de GitHub.
- ➕ Hacer push/pull de cambios.
- 🖥 Ejecutar comandos en servidores remotos de forma segura.

## 1 OBTENER LA URL SSH

En GitHub, entra al repositorio y haz clic en **Code** → **SSH**.



La URL SSH tiene este formato:  
`git@github.com:usuario/repositorio.git`

## 2 EJECUTAR EL CLONE

Abre tu terminal (PowerShell, Git Bash, Terminal, etc.) y ejecuta:

```
git clone git@github.com:usuario/repositorio.git
```

Si es la primera vez que te conectas a GitHub desde este equipo, SSH verificará la identidad del servidor antes de continuar.

## 3 VERIFICAR EL FINGERPRINT DE GITHUB (Primera conexión)

SSH muestra la huella (fingerprint) del servidor GitHub. Debes verificar que sea correcta.

```
The authenticity of host 'github.com (140.82.114.4)'
can't be established.
ED25519 key fingerprint is
SHA256:+DiY3wvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
Are you sure you want to continue connecting
(yes/no/[fingerprint])?  - 00000000.c0ec
```

Compara el fingerprint con el oficial de GitHub:  
**SHA256:+DiY3wvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU**  
(documentado en [docs.github.com](https://docs.github.com))  
Si coincide, escribe: **yes** y presiona **Enter**.

SSH guardará el servidor en: `~/.ssh/known_hosts`  
(ya no volverá a preguntarte por GitHub).

## 4 AUTENTICACIÓN CON TU CLAVE SSH

Después de aceptar el fingerprint, SSH usa tu clave privada para demostrar a GitHub que eres tú.



Si tu clave SSH está correctamente configurada y registrada en GitHub, el acceso se concede automáticamente.

## 5 CLONADO EXITOSO

Si todo está correcto, el repositorio se clona en tu equipo.

```
Cloning into 'repositorio'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (85/85), done.
remote: Total 123 (delta 14), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (123/123), 45.12 KiB | 1.23 MiB/s, done.
Resolving deltas: 100% (14/14), done.
```

El repositorio ya está en tu máquina y listo para trabajar.  
`cd repositorio` y comienza a usar Git.

## 6 ¿QUÉ HACER SI OCURRE UN ERROR?

Ejemplo del error mostrado:

```
git@github.com: Permission denied (publickey).
fatal: Could not read from remote repository.
```

Please make sure you have the correct access rights and the repository exists.

✗ Causa más común: tu clave SSH no está configurada o no está registrada en GitHub.

✓ Qué verificar:

- Que tienes una clave SSH generada (`ssh -T git@github.com`).
- Que la clave pública está agregada en tu cuenta de GitHub (Settings → SSH and GPG keys).
- Que estás usando la URL SSH correcta del repositorio.
- Que tienes permisos de acceso al repositorio.

## CONFIGURACIÓN INICIAL (SI AÚN NO TIENES CLAVE SSH)

- 1 Verifica si ya tienes claves:  
`ls -al ~/.ssh`
- 2 Genera una nueva clave SSH:  
`ssh-keygen -t ed25519 -C "tu_email@ejemplo.com"`
- 3 Inicia el agente SSH:  
`eval "$(ssh-agent -s)"`
- 4 Agrega tu clave privada al agente:  
`ssh-add ~/.ssh/id_ed25519`
- 5 Copia tu clave pública y agrégala a GitHub:  
`cat ~/.ssh/id_ed25519.pub`  
(Copia el contenido y pégalo en GitHub → Settings → SSH and GPG keys → New SSH key)

## FLUJO COMPLETO (RESUMEN)



## BUENAS PRÁCTICAS

- ✓ Nunca compartas tu clave privada.
- ✓ Usa una clave diferente para cada cuenta.
- ✓ Protege tu clave privada con una buena contraseña o passphrase.
- ✓ Mantén tu clave pública registrada en GitHub.
- ✓ Puedes tener múltiples claves SSH para distintos usos.

## TIPOS DE FINGERPRINT EN SSH

- 🛡 Fingerprint del servidor (GitHub)  
Sirve para verificar que te conectas al servidor correcto.
- 🔑 Fingerprint de tu clave SSH  
Identifica tu clave pública en GitHub. No se usa en el proceso de conexión, sino para gestión de claves.



**¡IMPORTANTE!** La primera conexión por SSH siempre verifica el fingerprint del servidor. Si no coincide, NO continúes: podría ser un ataque (man-in-the-middle).

🛡 SSH te da seguridad, comodidad y control sobre tus conexiones.



## ¿PROBLEMAS?

Usa: `ssh -T git@github.com` para probar tu conexión.  
Debe mostrar: `Hi usuario! You've successfully authenticated...`